

Practice Problem:

Write a function `multiply_by_2(a)` that takes one integer as input and returns the result of multiplying it by 2.

```
#include <iostream>
using namespace std;

int multiply_by_2(int a) {
    return a * 2;
}

int main() {
    int num = 5;
    cout << "5 multiplied by 2 is " << multiply_by_2(num) << endl;
    return 0;
}
```



Practice Problem:

Write a function `find_max_of_three(a, b, c)` that returns the maximum of three numbers.

```
#include <iostream>
using namespace std;

int find_max_of_three(int a, int b, int c) {
    return max(a, max(b, c));
}

int main() {
    cout << "The maximum of 3, 5, and 2 is " << find_max_of_three(3, 5, 2)
    return 0;
}
```

Practice Problem:

Write a function `square_of_sum(a, b)` that calculates the square of the sum of two numbers.

```
#include <iostream>
using namespace std;

int square_of_sum(int a, int b) {
    int sum = a + b;
    return sum * sum;
}

int main() {
    cout << "The square of the sum of 3 and 2 is " << square_of_sum(3, 2)
    return 0;
}
```



Practice Problem:

Write a function `fahrenheit_to_celsius(f)` that converts Fahrenheit to Celsius.

```
#include <iostream>
using namespace std;

float fahrenheit_to_celsius(float f) {
    return (f - 32) * 5 / 9;
}

int main() {
    cout << "32°F is equal to " << fahrenheit_to_celsius(32) << "°C" << endl;
    return 0;
}
```



Practice Problem:

Write a function `is_prime_easy(n)` that checks if a number is prime (only for $n > 1$).

```
#include <iostream>
using namespace std;

bool is_prime_easy(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    cout << "7 is " << (is_prime_easy(7) ? "prime" : "not prime") << endl;
    return 0;
}
```



Practice Problem:

Write a function `factorial_easy(n)` to find the factorial of a number.

Copy Edit

```
#include <iostream>
using namespace std;

int factorial_easy(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    cout << "Factorial of 4 is " << factorial_easy(4) << endl;
    return 0;
}
```



Practice Problem:

Write a function `sum_of_digits(n)` that calculates the sum of the digits of a number.

```
#include <iostream>
using namespace std;
```

Copy Edit

```
int sum_of_digits(int n) {
    int sum = 0;
    while (n != 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}
```

```
int main() {
    cout << "Sum of digits in 123 is " << sum_of_digits(123) << endl;
    return 0;
}
```



Practice Problem:

Write a function `gcd_of_three(a, b, c)` that calculates the GCD of three numbers.

```
#include <iostream>
using namespace std;

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int gcd_of_three(int a, int b, int c) {
    return gcd(gcd(a, b), c);
}

int main() {
    cout << "GCD of 12, 18, and 24 is " << gcd_of_three(12, 18, 24) << endl;
```

Practice Problem:

Write a function `reverse_number_even_only(n)` that reverses a number if it is even.

Solution:

```
#include <iostream>
using namespace std;

int reverse_number_even_only(int n) {
    if (n % 2 != 0) return n; // Return the number as-is if odd
    int reversed = 0;
    while (n != 0) {
        reversed = reversed * 10 + (n % 10);
        n /= 10;
    }
    return reversed;
}

int main() {
    cout << "Reversed number of 246 is " << reverse_number_even_only(246) << endl;
    return 0;
}
```

Practice Problem:

Write a function `add_and_double(a, b)` that adds two numbers and doubles the result.

1. String Input and Output

Problem:

Write a program that takes a string as input from the user and prints it in the format:

You entered: <input string>

Use the `getline()` function to handle spaces in the input.

Expected Output:

If the user enters Hello World, the program should output:

You entered: Hello World

2. Compare Two Strings

Problem:

Write a program that takes two strings as input and checks if they are equal. Print "Strings are equal" if they match; otherwise, print "Strings are not equal".

Expected Output:

For inputs apple and apple, output:

Strings are equal

For inputs apple and orange, output:
Strings are not equal

Hint: Use the == operator to compare strings in C++.

3. Convert String to Uppercase

Problem:

Write a function to_uppercase(string str) that converts all characters of a string to uppercase. Use this function to print the uppercase version of a user-provided string.

Expected Output:

If the user enters hello world, output:
Uppercase: HELLO WORLD

Hint: Use a loop or the transform() function from the <algorithm> library.

4. Count Occurrences of a Character

Problem:

Write a program that takes a string and a character as input, then counts how many times that character appears in the string.

Example Input:

String: programming
Character: g

Expected Output:

The character 'g' appears 2 times.

Hint: Use a loop to iterate through the string and count occurrences of the character.

5. Check for Substring

Problem:

Write a program that takes two strings as input. Check if the second string is a substring of the first. If it is, print "Substring found"; otherwise, print "Substring not found".

Example Input:

String 1: programming
String 2: gram

Expected Output:

Substring found

Hint: Use the find() function from the <string> library to check for substrings.

1. String Input and Output

Problem: Take a string as input and print it.

Solution:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string input;
    cout << "Enter a string: ";
    getline(cin, input); // Use getline to read the full line
    cout << "You entered: " << input << endl;
    return 0;
}
```



2. Compare Two Strings

Problem: Check if two strings are equal.

Solution:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1, str2;
    cout << "Enter the first string: ";
    cin >> str1;
    cout << "Enter the second string: ";
    cin >> str2;

    if (str1 == str2) {
        cout << "Strings are equal" << endl;
    } else {
        cout << "Strings are not equal" << endl;
    }
    return 0;
}
```



3. Convert String to Uppercase

Problem: Convert all characters in a string to uppercase.

Solution:

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

string to_uppercase(string str) {
    for (char &c : str) {
        c = toupper(c); // Convert each character to uppercase
    }
    return str;
}

int main() {
    string input;
    cout << "Enter a string: ";
    getline(cin, input);

    cout << "Uppercase: " << to_uppercase(input) << endl;
    return 0;
}
```



4. Count Occurrences of a Character

Problem: Count how many times a character appears in a string.

Solution:



```
#include <iostream>
#include <string>
using namespace std;

int count_character(string str, char c) {
    int count = 0;
    for (char ch : str) {
        if (ch == c) count++;
    }
    return count;
}

int main() {
    string input;
    char character;
    cout << "Enter a string: ";
    cin >> input;
    cout << "Enter a character to count: ";
    cin >> character;

    cout << "The character '" << character << "' appears "
         << count_character(input, character) << " times." << endl;
    return 0;
}
```



5. Check for Substring

Problem: Check if one string is a substring of another.

Solution:

```
#include <iostream>
#include <string>
using namespace std;

bool is_substring(string str1, string str2) {
    return str1.find(str2) != string::npos; // Check if str2 is found in str1
}

int main() {
    string str1, str2;
    cout << "Enter the first string: ";
    cin >> str1;
    cout << "Enter the second string: ";
    cin >> str2;

    if (is_substring(str1, str2)) {
        cout << "Substring found" << endl;
    } else {
        cout << "Substring not found" << endl;
    }
    return 0;
}
```